

Track A — Full Pipeline (Stage A → Stage B) Local Guide

This document explains every step, toggle, and output for the Track A pipeline that prepares proposals and head manifests for downstream Track B/C. Edit the in-file configs (no CLI needed) and keep everything local.

1) Prerequisites & Layout

- Frames live under `local_extraction/v2/extracted_frames/<uid>/<frame:07d>.jpg` .
- Optional YOLO GT labels for oracle/recall live under `local_extraction/v2/yolo_labels_540/{clips|videos}/<uid>/<frame:07d>.txt` .
- Optional head manifests (semantic labels) live under `local_extraction/v2/manifests/head_*_{clip|video}.json[1]` .
- Runs are written under `local_extraction/runs/Track_A/` .
- Stage A script: `local_extraction/trackA/trackA_stageA/trackA_stageA.py`
- Stage B script: `local_extraction/trackA/trackA_stageB/trackA_stageB.py`

Quick check (PowerShell):

```
ls local_extraction/v2/extracted_frames | measure
ls local_extraction/v2/manifests
```

2) Stage A — Detector Proposals (trackA_stageA.py)

Goal: produce candidate boxes per frame (either from YOLO inference or oracle labels) and a summary file.

Key toggles (edit inside the script)

- `RUN_NAME` : used to create `runs/Track_A/<RUN_NAME>/` .
- `VERSION` : usually `v2` (adjust if your frames live elsewhere).
- `SPACE` : `clips` or `videos` (affects oracle label path).
- `DETECTION_MODE` : `oracle` (uses GT labels) or `yolo` (runs Ultralytics YOLO locally).
- `K` : keep top-K boxes per image.
- `LAST_FRAME_ONLY` : `True` to process only the final frame per UID; `False` to process all frames.
- `MAX_IMAGES` : cap total images for quick smoke tests.
- YOLO-specific: `YOLO_WEIGHTS` , `YOLO_IMGSZ` , `YOLO_CONF` , `YOLO_IOU` .
- `SAVE_PER_IMAGE_CSV` : `True` to emit one CSV per image alongside the JSONL.

How to run

```
python local_extraction/trackA/trackA_stageA/trackA_stageA.py
```

The script reads frames, generates proposals, and writes outputs under

`local_extraction/runs/Track_A/<RUN_NAME>/` .

Outputs (Stage A run folder)

- `candidates.jsonl` : one line per image, e.g.:

```
{"uid": "00be...1705", "frame": 99, "boxes":
[{"x1": 161.8, "y1": -0.3, "x2": 373.9, "y2": 244.8, "conf": 1.0, "cls": 26}]}
```

- Optional per-image CSVs in `candidates/` if `SAVE_PER_IMAGE_CSV=True` .
- `summary.json` : counts and settings, e.g.:

```
{
  "images": 500,
  "avg_boxes_per_image": 4.2,
  "mode": "oracle",
  "K": 10,
  "last_frame_only": true,
  "runtime_sec": 123.4
}
```

Internal flow (what the code does)

1. Build run dir (`runs/Track_A/<RUN_NAME>/`).
2. Collect frames (last frame per UID by default; or all frames).
3. For each image:
 - Oracle mode: read GT label txt, convert YOLO `cx,cy,w,h` → `x1,y1,x2,y2` , keep top-K.
 - YOLO mode: run Ultralytics with your weights/thresholds, keep top-K by confidence.
 - Append to `candidates.jsonl` ; optionally write a per-image CSV.
4. After processing: compute counts/averages and write `summary.json` .

Troubleshooting Stage A

- **No boxes in output:** confirm `DETECTION_MODE` matches available inputs (oracle requires label txt files; YOLO requires weights + `ultralytics` installed).
- **Wrong frames picked:** check `LAST_FRAME_ONLY` and `MAX_IMAGES` ; inspect the run folder for sampled frames.
- **Boxes in wrong scale:** ensure label txts match the frame resolution; oracle conversion assumes normalized YOLO format.
- **Performance:** reduce `MAX_IMAGES` , set `LAST_FRAME_ONLY=True` , lower `K` , or switch to oracle for quick recall checks.

3) Stage B — Crops & Head Manifests (trackA_stageB.py)

Goal: take Stage A proposals, crop ROIs, optionally measure recall against GT, and emit head manifests for Track B.

Key toggles (edit inside the script)

- Inputs:
 - `CANDIDATES_JSONL` : path to Stage A `candidates.jsonl` (auto-detects latest if `None`).
 - `FRAMES_ROOT` , `LABELS_ROOT` , `LABEL_SPACE` , `EVAL_WITH_LABELS` , `IOU_THRESH` .
- Cropping:
 - `KEEP_TOP_N` : keep first N candidates per image (None = all).
 - `CROP_SIZE` : (W,H) to resize each crop; `None` keeps native size.
- Limits:
 - `MAX_IMAGES` , `DEMO_MODE` , `DEMO_N` , `PRINT_PROGRESS` .
- Head manifests:
 - `USE_CLIP_MANIFEST` : choose `head*_clip.json` vs `head*_video.json` .
 - `HEAD_TRAIN_MANIFEST` , `HEAD_VAL_MANIFEST` : explicit paths if needed.
 - `WRITE_HEAD_TRAIN_VAL` : emit merged `head_train.jsonl` / `head_val.jsonl` in the run dir.
 - `WRITE_SEMANTICS_IN_MANIFEST` : include semantic fields in per-crop manifest rows.
 - `WRITE_BACK_TO_STAGEA_SUMMARY` : write recall numbers back into the Stage A `summary.json` .
- Run dir: `RUN_DIR` defaults to `runs/Track_A/trackA_stageB_<timestamp>/` .

How to run

```
python local_extraction/trackA/trackA_stageB/trackA_stageB.py
```

Outputs (Stage B run folder)

- `crops/` : cropped and optionally resized ROI images.
- `manifest.jsonl` / `manifest.csv` : per-crop rows with:
 - `uid` , `frame` , `image_path` , `crop_path` , `bbox` , `bbox_clamped` , `bbox_norm` , `W` , `H` .
 - If semantics are available: `is_positive` , `verb_id` , `noun_id` , `ttc` , `ttc_bin` .
- `head_train.jsonl` / `head_val.jsonl` (if `WRITE_HEAD_TRAIN_VAL=True` and manifests exist):
 - One record per frame, grouped candidates with semantic labels ready for Track B.
- Recall stats (if `EVAL_WITH_LABELS=True`):
 - Recall@K = (# frames with ≥ 1 hit) / (# frames with ≥ 1 GT).
 - Mean best IoU, histogram; optionally mirrored into Stage A `summary.json` .

Sample head manifest record

```
{
  "uid": "03abc",
  "frame": 120,
  "image": "local_extraction/v2/extracted_frames/03abc/0000120.jpg",
  "candidates": [
    {"bbox": [123.4, 56.7, 200.1, 180.0], "is_positive": 1, "noun_id": 42,
    "verb_id": 7, "ttc": 0.83, "ttc_bin": 1},
    {"bbox": [10.0, 10.0, 40.0, 50.0], "is_positive": 0, "noun_id": -1,
    "verb_id": -1, "ttc": 2.5, "ttc_bin": 3}
  ]
}
```

Internal flow (what the code does)

1. Resolve `CANDIDATES_JSONL` (latest Stage A if None).
2. Build `RUN_DIR` and `crops/`.
3. Load optional head manifests for semantics (clip or video variant).
4. Iterate candidates:
 - Load frame, clamp boxes, optionally limit to `KEEP_TOP_N`.
 - Crop and resize each ROI to `CROP_SIZE`; save under `crops/`.
 - Emit per-crop manifest rows; attach semantics if available.
5. If semantics + `WRITE_HEAD_TRAIN_VAL=True`, merge into grouped `head_train.jsonl` / `head_val.jsonl`.
6. If `EVAL_WITH_LABELS=True`, compute recall@K and IoU stats; optionally patch Stage A summary.

Troubleshooting Stage B

- **Empty crops/manifest:** check `CANDIDATES_JSONL` path; ensure Stage A run exists and has boxes.
- **Low recall:** inspect label source (`LABEL_SPACE` , `IOU_THRESH`); confirm GT label folder matches frames.
- **Missing semantics:** verify head manifests exist and `USE_CLIP_MANIFEST` is set to the right variant.
- **Cropping errors:** ensure frames exist at `FRAMES_ROOT`; look for permission or path typos.

4) Hand-off to Track B (and Track C)

- Track B auto-discovers the latest Stage B run via `latest_stageB_run` inside `trackB_dataset.py`.
- The Stage B `head_train.jsonl` / `head_val.jsonl` become the training/validation manifests for Track B.
- Track C pruning uses the same Stage B manifests and the Track B checkpoint; no changes needed in Track A once manifests are produced.

5) Quick Recipes

- **Fast oracle smoke test (recall@K focus):**
 - Stage A: `DETECTION_MODE='oracle'` , `LAST_FRAME_ONLY=True` , `K=5` , small `RUN_NAME` .

- Stage B: point `CANDIDATES_JSONL` to that run, set `EVAL_WITH_LABELS=True` , `KEEP_TOP_N=5` .
 - **YOLO proposals for real training:**
 - Stage A: `DETECTION_MODE='yolo'` , set weights/thresholds, `K=10` .
 - Stage B: `USE_CLIP_MANIFEST=True` , `WRITE_HEAD_TRAIN_VAL=True` , leave `CROP_SIZE=(256,256)` .
 - **Semantics off (binary only):**
 - Stage B: set `WRITE_HEAD_TRAIN_VAL=False` ; Track B will still see boxes/ttc if present but can run binary+TTC.
 - **Throttle runtime:**
 - Stage A: set `MAX_IMAGES` and `LAST_FRAME_ONLY=True` .
 - Stage B: set `MAX_IMAGES` or `DEMO_MODE=True` with `DEMO_N` .
-

6) Outputs Cheat-Sheet (by folder)

- `runs/Track_A/StageA_*` (or custom `RUN_NAME`):
 - `candidates.jsonl` , optional `candidates/*.csv` , `summary.json` .
 - `runs/Track_A/trackA_stageB_*` :
 - `crops/` , `manifest.jsonl` / `.csv` , `head_train.jsonl` , `head_val.jsonl` , recall stats.
 - `runs/Track_B/` (downstream, for reference):
 - `checkpoints/` , `metrics/` , `predictions/` , `overlays/` , `plots/` .
 - `runs/Track_C/` (downstream pruning, for reference):
 - `metrics/` and plots for RGTP experiments.
-

7) Field Reference

- Candidates (Stage A):
 - `boxes` : list of `{x1,y1,x2,y2,conf,cls}` ; coordinates in pixels.
 - Crops (Stage B):
 - `bbox` : original box; `bbox_clamped` : clipped to frame; `bbox_norm` : normalized `[0,1]` .
 - `crop_path` : saved ROI; `W` , `H` : frame size.
 - Head manifests (Stage B → Track B):
 - `is_positive` (0/1), `noun_id` , `verb_id` , `ttc` , `ttc_bin` , `bbox` .
-

8) Common Pitfalls & Fixes

- **Stage A finds zero images:** verify `VERSION` and frame path; ensure frames are `.jpg` with zero-padded names.
 - **YOLO import error:** install `ultralytics` locally and confirm weights path exists.
 - **Stage B recall is zero:** make sure GT labels are in the same `SPACE` (`clips` vs `videos`) and frames align.
 - **Manifest mixing clip/video:** set `USE_CLIP_MANIFEST` consistently; delete stale files in the run dir before rerunning.
 - **Large run folders:** disable per-image CSVs, reduce `K` , and clean old runs under `runs/Track_A/` .
-

9) Integration Notes for Track B/C

- Track B expects head manifests with `is_positive` and TTC; noun/verb/TTC-bin fields enable multi-task heads and N/N+V/N+ δ metrics.
 - If you only need next-active + TTC, you can omit semantics (Track B falls back to binary + TTC regression).
 - Track C pruning reuses the same manifests and only needs the Track B checkpoint; no changes to Track A once manifests are ready.
-

10) Versioning & Repro Tips

- Always set a descriptive `RUN_NAME` (include mode, K, and date), e.g., `stageA_oracle_K5_2025-11-26`.
 - Keep a small text note in the run folder describing any deviations (e.g., different `IOU_THRESH`).
 - For reruns, copy the exact toggles used in Stage A/B into your experiment log to make Track B/C results reproducible.
-

11) File Map (code)

- `trackA_stageA/trackA_stageA.py` — Stage A runner (oracle/YOLO proposals).
 - `trackA_stageA/oracle_k_sweep.py` — sweep K for oracle recall.
 - `trackA_stageA/sweep_k_metrics.py` — analyze K vs recall metrics.
 - `trackA_stageB/trackA_stageB.py` — Stage B runner (crops + head manifests).
 - `trackA_stageB/san_stageB.py` — sanity checks on Stage B outputs.
-

12) Minimal End-to-End Checklist

1. Verify frames exist under `local_extraction/v2/extracted_frames/`.
2. Run Stage A (oracle or YOLO) → confirm `candidates.jsonl` + `summary.json`.
3. Run Stage B → confirm `manifest.jsonl` and `head_train/val.jsonl`.
4. Point Track B to the Stage B run (auto-discovery works by default).
5. Train Track B → check `runs/Track_B/checkpoints/trackB_best.pt` and metrics.
6. (Optional) Run Track C pruning and compare metrics/plots.

This guide is intentionally verbose to serve as a single source of truth for Track A. Trim sections locally if you prefer a shorter version.

Track B — Complete Local Pipeline Guide (Stage B Head, Multi-Task, Priors, Evaluation)

This document is intentionally exhaustive (~500 lines) so you can run, audit, and extend Track B without diving back into the code. It covers data assumptions, modules, configs, training/eval flows, outputs, troubleshooting, and reference tables.

0. TL;DR Quick Path

- Prepare inputs: Stage B manifests (`head_train.jsonl` , `head_val.jsonl`) and frames under `local_extraction/v2/extracted_frames/<uid>/` .
 - Train: edit `TrainConfig` in `trackB_train_loader.py` , then run `python local_extraction/trackB/trackB_train_loader.py` .
 - Eval: edit `EvalConfig` in `trackB_eval.py` , then run `python local_extraction/trackB/trackB_eval.py` .
 - Outputs: checkpoints in `runs/Track_B/checkpoints/` , metrics/preds/overlays in `runs/Track_B/{metrics,predictions,overlays/val}` ; plots in `runs/Track_B/plots/` .
 - Multi-task (noun/verb/TTC-bin) is controlled by `use_multi_task_labels` / `use_ttc_bins` in `TrainConfig` ; hotspot/CLIP are eval-time only.
-

1. Repository Layout (Track B)

- `trackB_dataset.py` — Dataset + collate; manifest discovery; TTC stats; multi-task labels.
 - `trackB_tokenizer.py` — Backbone builder, transforms, frame/video tokenization, ROI pooling.
 - `trackB_fusion.py` — Frame-Guided Temporal Pooling + dual cross-attention blocks.
 - `trackB_head.py` — Classification + TTC regression, optional noun/verb/TTC-bin heads.
 - `trackB_train_loader.py` — Main training loop with validation, early stopping, checkpointing.
 - `trackB_train_demo.py` — Minimal/demo trainer with synthetic fallback.
 - `trackB_eval.py` — Standalone evaluation (metrics, predictions, overlays).
 - `trackB_metrics.py` — Accuracy, AP, TTC MAE helpers.
 - `trackB_plots.py` — Aggregates metrics JSON into PNG + TSV.
 - `trackB_manifest_validator.py` — Coverage/invalid logging for manifests.
 - `trackB_show_samples.py` , `trackB_show_config.py` — Inspection helpers.
 - `trackB_tests.py` — Small sanity tests.
-

2. Data & Manifests Assumptions

- Frames: `local_extraction/v2/extracted_frames/<uid>/<frame:07d>.jpg` .
 - Stage B manifests: `head_train.jsonl` / `head_val.jsonl` from Track A Stage B run or `local_extraction/v2/manifests/` .
 - Candidate fields expected (per record, grouped by frame):
 - `bbox` : `[x1, y1, x2, y2]` in pixel coords.
 - `is_positive` : 0/1 next-active flag (falls back to `cls==1` if absent).
 - `verb_id` , `noun_id` : ints or `-1` if missing.
 - `ttc` : seconds (float); `ttc_bin` optional.
 - Dataset will compute `ttc_bin` if missing via thresholds `(0.5, 1.0, 2.0)` .
 - If manifests are missing, synthetic records are generated (config-dependent).
-

3. Tokenization Pipeline (`trackB_tokenizer.py`)

- `TokenizerConfig` holds device, image size, time length/stride, normalization.
- `build_backbone` returns a CLIP-like encoder (512-dim tokens).

- `build_transform` returns preprocessing matching the backbone.
 - `list_uid_frames` enumerates frames per UID.
 - `sample_window_ending_at` selects a temporal window ending at the target frame.
 - `image_grid_tokens(path, backbone, transform, cfg) → (N,512)` tokens + grid hw.
 - `video_grid_tokens(window, backbone, transform, cfg) → (T,N,512)` tokens.
 - `roi_pool_tokens_mean(hw, tokens, box, image_size) →` pooled token for an ROI.
 - Keep `num_workers=0` unless you move tokenization out of the dataset.
-

4. Fusion Module (trackB_fusion.py)

- `FusionConfig` : `dim` , `layers` , `heads` , dropout.
 - `FGTP` : Frame-Guided Temporal Pooling projects video tokens onto the last-frame grid.
 - `DualCrossAttention` : symmetric image↔video cross-attention block (lightweight).
 - `TrackBFusion` : FGTP + stacked cross-attention layers; outputs `(fused_img, fused_vid)` each `(B,N,dim)` .
 - Default: `dim=256`, `layers=2`, `heads=8` (tuned for speed/simplicity).
-

5. Head Module (trackB_head.py)

- `HeadConfig` : `dim` , `num_classes` (≥ 2), `hidden` , `dropout` , optional `num_noun_classes` , `num_verb_classes` , `num_ttc_bins` , `ttc_mode` (`reg` or `bin`).
 - `TrackBHead` outputs:
 - `cls_logits` : `(B,N,K_next)` next-active logits.
 - `ttc` : `(B,N,1)` continuous TTC regression.
 - Optional: `noun_logits` , `verb_logits` , `ttc_bin_logits` if enabled.
 - Multi-task is gated by dataset label availability and `TrainConfig` toggles.
-

6. Dataset (trackB_dataset.py)

- Init args: `frames_root` , `manifests_root` , `manifest_path` , `tokenizer_cfg` , `candidate_limit` , `normalize_ttc` , `synthetic_if_empty` , `cache_dir` , `seed` .
- Manifest discovery: tries `head_train*.json[1]` (or `head_val*.json[1]`) in provided root; Stage B run auto-discovery via `latest_stageB_run` .
- Parsing:
 - Accepts flexible keys (`uid|video_uid` , `frame|frame_idx|frame_index` , `frame_name|image` , `candidates|boxes|objects`).
 - Converts per-candidate dict/list into normalized fields: `bbox` , `cls` , `is_positive` , `verb_id` , `noun_id` , `ttc` , `ttc_bin` .
 - TTC stats computed (mean/std/min/max); `ttc_norm` added if `normalize_ttc=True` .
- `__getitem__` :
 - Resolves frame path (absolute or via UID folder).
 - Builds `img_tokens` (`N,512`) and `vid_tokens` (`T,N,512`).
 - Applies `candidate_limit` by random subsample.
 - Returns dict with tokens, `hw` , `image_size` , `bboxes` , labels, `is_positive` , `gt_noun_id` , `gt_verb_id` , `gt_ttc` , `gt_ttc_bin` , `ttc` , `ttc_norm` , `valid` .

- Collate (`trackB_collate`): returns `{'samples': [...], 'valid': True}` preserving variable candidate counts.
- QA: invalid records logged to `runs/Track_B/cache/invalid_records.csv` ; TTC stats saved in cache.

7. Training Config (trackB_train_loader.py — TrainConfig fields)

- Core:
 - `mode` : 'main' or 'demo' .
 - `epochs` , `batch_size` , `lr` , `min_lr` , `warmup_epochs` .
 - `candidate_limit` , `label_smoothing` , `normalize_ttc` , `amp` .
- Validation:
 - `eval_every` , `early_stopping_patience` , `monitor_metric` (`mAP` / `accuracy` / `ttc_mae`), `greater_is_better` auto-set for `ttc_mae` .
 - `save_best_checkpoint` , `save_epoch_checkpoints` .
 - `val_manifest` : optional override; otherwise discovered.
- Data:
 - `train_manifest` , `stageB_run` override, `time_len` , `time_stride` via `TokenizerConfig` .
- Multi-task:
 - `use_multi_task_labels` (enable noun/verb heads).
 - `use_ttc_bins` (enable TTC-bin head and CE loss).
 - Loss weights: `loss_w_next` , `loss_w_noun` , `loss_w_verb` , `loss_w_ttc` .
- Demo:
 - `demo_steps` , `demo_variant` , `demo_use_real_labels` , `demo_manifest` .

8. Training Pipeline (trackB_train_loader.py — main mode)

- 1. Build dataset `ds` (train manifest) and optional `ds_val` (val manifest).
- 2. Infer vocab sizes from parsed candidates:
 - Next-active classes: max label + 1, min 2.
 - Noun/verb vocab from observed IDs when `use_multi_task_labels=True` .
 - TTC bins from observed when `use_ttc_bins=True` .
- 3. Instantiate modules:
 - `projector` : Linear 512→TOKEN_DIM (256).
 - `fusion` : `TrackBFusion` (dim=256, layers=2, heads=8).
 - `head` : `TrackBHead` with inferred class counts and `ttc_mode` (`reg` or `bin`).
- 4. Optimizer: Adam over projector+fusion+head; scaler for AMP if enabled.
- 5. Training loop per epoch:
 - Shuffle loader; for each batch:
 - For each sample: project tokens, fuse, ROI-pool per candidate → tensors (N_c,256).
 - Pad to max candidates in batch; build masks.
 - Forward head; compute losses:
 - Next-active CE on `is_positive` (or `labels` fallback).
 - TTC: SmoothL1 (reg) or CE (bin) on `gt_ttc_bin` if enabled.

- Noun/verb CE on positives when multi-task enabled.
 - Weighted sum via `loss_w*`.
 - Backprop + optimizer step; optional AMP.
 - Log per-epoch avg losses (`next` , `ttc` , `noun` , `verb`).
 - 6. Validation (every `eval_every` epochs):
 - `_evaluate_loader` computes metrics on `ds_val` (same as eval script subset).
 - Track `best_val` on `monitor_metric` ; save best checkpoint(s).
 - 7. Checkpoints:
 - Epoch checkpoints if `save_epoch_checkpoints=True` .
 - Best checkpoint: `trackB_best_<metric>_<score>_<ts>.pt` + alias `trackB_best.pt` .
 - Summary JSON alongside best checkpoint: `trackB_best_<metric>_<score>_<ts>_summary.json` and `trackB_best_summary.json` .
 - Final checkpoint: `trackB_final_<ts>.pt` .
 - 8. Post-train quick sanity: prints one batch's preds/acc/ttc.
-

9. Demo Pipeline (trackB_train_demo.py)

- Smaller, single-process loop for debugging.
 - Toggle `DemoConfig.use_real_labels` to use manifests or synthetic.
 - Good for checking token shapes, loss computation, and overfit-on-small-batch behavior.
-

10. Evaluation Script (trackB_eval.py)

- Config (`EvalConfig`):
 - Paths: `frames_root` , `manifests_root` , `trackA_runs_root` .
 - `val_manifest` : optional override (else discovery order: `head_val_video.json[l]`, `head_val_clip.json[l]`, `val.json[l]`).
 - `checkpoint_path` : optional override (else latest `trackB_final_*.pt` or `trackB_best.pt`).
 - Model: `token_dim` , `num_classes` (fallback), `fusion_layers` .
 - Data: `batch_size` , `candidate_limit` , `normalize_ttc` .
 - Outputs: `topk_overlay` , `save_overlays` .
 - Multi-task eval: `ttc_mode` (`reg` / `binned`), `iou_thresh` .
 - Priors: `use_hotspot_priors` , `hotspot_prior_path` , `hotspot_alpha` .
 - CLIP re-rank: `use_clip_rerank` , `clip_weight` , `clip_model` , `noun_label_path` .
- Flow:
 1. Discover val manifest and checkpoint; load projector/fusion/head; infer class counts from checkpoint weights.
 2. Build `TrackBDataset` (val) with tokenizer; DataLoader (num_workers=0).
 3. For each sample:
 - Tokenize, project, fuse, ROI-pool per candidate.
 - Forward head; apply optional hotspot/CLIP re-scoring to next-active probs.
 - Collect logits/labels, TTC (denorm), semantics if available.
 - Compute frame-level top-1 and top-5 metrics (N/N+V/N+ δ /All) when noun/verb present.
 - Save overlay images if enabled (top-K per frame).

4. Aggregate metrics:

- Candidate-level: accuracy, mAP, `ttc_mae_seconds` .
- Frame-level: `N_mAP` , `Nv_mAP` , `N_delta_mAP` , `All_mAP` .
- Top-5 hit/AP metrics; per-noun/verb accuracy breakdowns.

5. Write outputs:

- `metrics/metrics_val_<ts>.json` (metrics only).
 - `metrics/metrics_val_<ts>_summary.json` (metrics + eval config).
 - Predictions CSV/JSONL under `predictions/` .
 - Overlays under `overlays/val/` .
 - CLI overrides: `--checkpoint` , `--val_manifest` , `--stageB_run` , `--ttc_mode` .
-

11. Metrics Definitions

- Candidate-level:
 - `accuracy` : next-active accuracy.
 - `mAP` : average precision (binary or macro over classes).
 - `ttc_mae_seconds` : mean absolute error on TTC (denormalized).
 - Frame-level (top-1):
 - `N_mAP` : noun matches on highest-prob candidate with IoU >= `iou_thresh` .
 - `Nv_mAP` : noun & verb match on top candidate with IoU >= threshold.
 - `N_delta_mAP` : noun & TTC bin match on top candidate with IoU >= threshold (predicted bin or bin-from-reg).
 - `All_mAP` : noun & verb & TTC bin match on top candidate with IoU >= threshold.
 - Frame-level (top-5):
 - Hit rates: `N_top5_acc` , `Nv_top5_acc` , `N_delta_top5_acc` , `All_top5_acc` .
 - AP (top-5 candidates only): `N_top5_mAP` , `Nv_top5_mAP` , `N_delta_top5_mAP` , `All_top5_mAP` .
 - Per-class breakdown:
 - `per_noun_stats` , `per_verb_stats` : count/correct/accuracy per taxonomy ID.
-

12. Hotspot Priors & CLIP Re-Rank (Eval-time only)

- Hotspot priors:
 - JSON format: `{"pairs": {"noun_id,verb_id": score}, "default": 0.0}` .
 - Applied as `score_final = (1 - alpha) * base + alpha * prior` .
 - Controls: `use_hotspot_priors=True` , set `hotspot_prior_path` , adjust `hotspot_alpha` .
 - CLIP re-rank:
 - Text prompts from `noun_label_path` (STA label JSON) matched to candidates.
 - Blends CLIP score with base next-active prob via `clip_weight` .
 - Controls: `use_clip_rerank` , `clip_weight` , `clip_model` .
 - Both only affect eval scoring, not training.
-

13. Checkpoints & Summaries

- Location: `local_extraction/runs/Track_B/checkpoints/` .

- Files:
 - `trackB_epochE_<ts>.pt` (if `save_epoch_checkpoints=True`).
 - `trackB_final_<ts>.pt` (always saved).
 - `trackB_best_<metric>_<score>_<ts>.pt` (best by monitor metric) + `trackB_best.pt` alias.
 - Summary JSONs: `trackB_best_<metric>_<score>_<ts>_summary.json` , `trackB_best_summary.json` .
 - Contents:
 - `projector` , `fusion` , `head` state_dicts.
 - `noun_id_list` , `verb_id_list` for mapping local head indices to global IDs.
 - `train_config` (serialized `TrainConfig`).
-

14. Plots & Comparisons

- `trackB_plots.py` :
 - Scans `metrics/metrics_val_*.json` .
 - Produces `plots/trackB_metrics_over_time.png` and `trackB_metrics_summary.tsv` .
 - Adds any new scalar metrics it finds.
 - Track C comparison:
 - `local_extraction/trackC/trackC_compare_metrics.py` compares latest Track B vs Track C metrics.
-

15. Running Commands (PowerShell examples)

- Train main:

```
python local_extraction\trackB\trackB_train_loader.py
```

- Eval standalone:

```
python local_extraction\trackB\trackB_eval.py --ttc_mode reg
```

- Fusion smoke:

```
python local_extraction\trackB\trackB_fusion.py
```

- Plot metrics:

```
python local_extraction\trackB\trackB_plots.py
```

16. Integration with Track A / Track C

- Track A Stage B produces `head_train.jsonl` / `head_val.jsonl` ; Track B auto-discovers latest via `latest_stageB_run` .
- Track C pruning consumes Track B checkpoint + same Stage B manifests; no changes required in Track B when evaluating pruning.

17. Troubleshooting & FAQs

- **No candidates parsed:** check manifest paths and keys (`candidates|boxes|objects`); run `trackB_manifest_validator.py` .
- **Empty val metrics:** ensure `val_manifest` resolves; set explicit path in `EvalConfig` or `TrainConfig` .
- **Cuda OOM in training:** reduce `batch_size` , `candidate_limit` , fusion layers; disable AMP if unstable.
- **Mismatch noun/verb IDs:** confirm manifest IDs are ints; `noun_id_list` / `verb_id_list` are inferred from data seen.
- **Priors/CLIP not changing scores:** ensure `use_hotspot_priors` / `use_clip_rerank` are True and paths exist; check `hotspot_alpha` / `clip_weight` .
- **ttc_mode confusion:** `reg` uses regression head; `binned` uses `ttc_bin_logits` if present; Track B will infer `ttc_mode` from checkpoint when bin head exists.
- **Slow DataLoader:** keep `num_workers=0` (dataset holds backbone); precompute tokens if scaling up.
- **Overlays missing:** set `save_overlays=True` and ensure frames exist; reduce `topk_overlay` if too many images.
- **Best checkpoint not saved:** verify `monitor_metric` is valid; check metric not NaN; ensure `save_best_checkpoint=True` .

18. Extended Field Reference (per-sample dict from Dataset)

- `uid` : string UID.
- `frame_path` : absolute path to the image.
- `img_tokens` : (N,512) float32.
- `vid_tokens` : (T,N,512) float32 (may fallback to single-frame).
- `hw` : (H_tokens, W_tokens) grid shape.
- `image_size` : (W,H) in pixels.
- `bboxes` : list of candidate boxes.
- `labels` : LongTensor[N] (legacy cls).
- `is_positive` : LongTensor[N] (next-active flag).
- `gt_noun_id` : LongTensor[N] (noun or -1).
- `gt_verb_id` : LongTensor[N] (verb or -1).

- `gt_ttc` : FloatTensor[N] (seconds).
 - `gt_ttc_bin` : LongTensor[N] (bin index).
 - `ttc` : FloatTensor[N] (alias of `gt_ttc`).
 - `ttc_norm` : FloatTensor[N] (z-scored if `normalize_ttc=True`).
 - `valid` : bool.
-

19. Extended Metric Interpretation

- Use `Nv_mAP` to judge noun+verb correctness on top candidate; sensitive to semantic heads.
 - `N_delta_mAP` reflects noun + TTC-bin agreement; useful when TTC bins are enabled.
 - Top-5 metrics capture recall when multiple candidates are plausible; useful for ambiguous scenes.
 - `ttc_mae_seconds` should be read alongside classification metrics; regression and binning can be compared by switching `ttc_mode`.
 - Per-noun/verb breakdowns help detect class imbalance or poor recall on specific actions.
-

20. Ablation Suggestions

- Fusion depth: layers {2,3,4}; dimension {192,256,320}; heads {4,8}.
 - Candidate pruning: vary `candidate_limit` in dataset and prune rate in Track C to study latency vs mAP.
 - TTC handling: compare regression vs binning (`use_ttc_bins=True`) on `N_delta_mAP` and `ttc_mae_seconds`.
 - Multi-task on/off: toggle `use_multi_task_labels` to see impact on N/N+V/N+ δ without changing data.
 - Priors: sweep `hotspot_alpha` and CLIP `clip_weight` to quantify gains on `Nv_mAP` in ambiguous scenes.
-

21. Serialization Details

- Checkpoints include `train_config` dict from `_config_to_dict` (scalar-friendly fields).
 - Best summaries include metrics snapshot and config; filenames mirror checkpoint name with `_summary`.
 - Eval summaries mirror metrics with serialized `EvalConfig` (paths converted to strings).
-

22. Stage-B Manifest Notes (consumed by Track B)

- Expected keys when writing your own:
 - `uid`, `frame` or `frame_idx`, `image` (path), `candidates`.
 - Each candidate: `x1,y1,x2,y2`, `is_positive`, `verb_id`, `noun_id`, `ttc`, optional `ttc_bin`.
 - Grouped per frame; Track B groups them if provided flat by image path in Stage B format.
 - Use consistent taxonomy IDs for noun/verb across train/val.
-

23. CLI vs In-File Config

- Training/eval scripts favor in-file config for reproducibility; only a few CLI overrides exist for eval.
 - Recommended: set paths and toggles directly in `TrainConfig` / `EvalConfig` and commit the files if you need reproducible runs.
-

24. Performance Tips

- Reduce `time_len` in tokenizer if temporal window is large and not critical.
 - Lower fusion layers or dim for faster runs; Monitor mAP drop.
 - Use AMP (`amp=True`) cautiously; verify stability.
 - For quick experiments, set `MAX_IMAGES` in Stage A/B and small `candidate_limit` in Track B.
-

25. Example Experiment Templates

- **Baseline single-task:** `use_multi_task_labels=False` , `use_ttc_bins=False` ; monitor `mAP` + `ttc_mae_seconds` .
 - **Full multi-task:** `use_multi_task_labels=True` , `use_ttc_bins=True` ; monitor `N_mAP` , `Nv_mAP` , `N_delta_mAP` .
 - **Hotspot+CLIP eval:** train with multi-task on; eval with `use_hotspot_priors=True` , `use_clip_rerank=True` , moderate alphas.
 - **Latency-aware:** pair Track B metrics with Track C pruning metrics to plot mAP vs prune rate.
-

26. Logging & Caching

- Cache dir: `runs/Track_B/cache/` holds TTC stats, coverage logs, invalid records CSV.
 - Training prints per-step loss (with postfixes) and per-epoch averages.
 - Validation prints metric dict per eval interval.
 - Eval prints metrics dict and paths written.
-

27. Overlay Rendering (Eval)

- Draws boxes with probability and TTC (pred + GT).
 - Colors cycle per class; top-K controlled by `topk_overlay` .
 - Saved under `runs/Track_B/overlays/val/` .
-

28. Predictions CSV/JSONL Schema

- Common fields:
 - `uid` , `frame` , `bbox` , `prob_pos` , `score_final` (after priors/CLIP), `prob_base` (pre-prior).
 - `pred_noun_id` , `pred_verb_id` , `pred_ttc_s` , `pred_ttc_bin` .
 - `gt_noun_id` , `gt_verb_id` , `gt_ttc_s` , `gt_ttc_bin` .
 - `hotspot_score` (if used), `clip_score` (if used).
 - JSONL mirrors CSV rows as dicts.
-

29. Dev Notes on Code Structure

- Minimal external deps: PyTorch, PIL, tqdm, (optional) CLIP/priors loaders.
 - Torch usage avoids JIT/custom kernels; ROI pooling is pure Python over token grids.
 - Tokenizer holds backbone; hence DataLoader workers must stay 0 unless refactored.
 - Functions are organized per concern: dataset parsing, fusion, head, metrics, train, eval.
-

30. Safety & Paths

- All scripts default to relative paths inside repo; no network calls.
 - Overwrite-safe: run dirs are timestamped; best/final names are stable aliases.
 - Avoid destructive commands; clean old runs manually if disk grows.
-

31. Minimal Code Walkthrough (Training)

- Dataset: yields tokens + labels per sample.
 - Batch loop:
 - For each sample: pool candidates → list of tensors.
 - Pad pooled tensors to (B, max_candidates, dim) with masks.
 - Head forward; losses masked where no candidate.
 - Sum weighted losses; backward; step optimizer.
 - Validation mirrors eval but without overlays/preds writing.
-

32. Minimal Code Walkthrough (Evaluation)

- Load checkpoint, infer head shapes.
 - Build val dataset/loader.
 - For each sample:
 - Project/fuse tokens; pool per candidate.
 - Head forward.
 - Optional priors/CLIP: adjust scores.
 - Collect logits/labels/TTC; compute frame-level semantics if heads present.
 - Save overlay if enabled.
 - Aggregate metrics; write JSON + summary + preds.
-

33. Extending Track B

- Swap backbone: change `build_backbone` and token dims; update projector/head dims.
 - Add heads: extend `HeadConfig` / `TrackBHead` ; add losses; update metrics if needed.
 - Precompute tokens: persist `img_tokens` / `vid_tokens` to disk and modify dataset to load them.
 - Multi-GPU: current loop is single-device; would need DistributedDataParallel + tokenization refactor.
-

34. Environment Assumptions

- Local filesystem; no network.
- CUDA optional; code runs on CPU (slower).

- Python 3; PyTorch installed; optional `ultraanalytics` for Stage A (outside Track B scope).
-

35. Cleanup & Housekeeping

- Remove stale runs in `runs/Track_B/` to save disk.
 - Keep best/final checkpoints; delete old epoch checkpoints if not needed.
 - Archive metrics/plots for reporting.
-

36. Glossary

- **FGTP**: Frame-Guided Temporal Pooling.
 - **N_mAP**: noun match AP (frame-level top-1).
 - **Nv_mAP**: noun+verb match AP (frame-level top-1).
 - **N_delta_mAP**: noun+TTC-bin match AP (frame-level top-1).
 - **All_mAP**: noun+verb+TTC-bin match AP (frame-level top-1).
 - **TTC**: Time-to-contact (seconds).
 - **Hotspot priors**: noun/verb pair prior scores blended into next-active probability.
 - **CLIP re-rank**: text-image similarity blended into next-active probability.
-

37. End-to-End Checklist (Concise)

- ☐ Stage B manifests present.
 - ☐ Frames accessible at `frames_root`.
 - ☐ TrainConfig set (multi-task, bins, monitor metric).
 - ☐ Run training; verify checkpoints and best summary JSON.
 - ☐ Run eval; verify metrics, preds, overlays, summary JSON.
 - ☐ Plot metrics; compare with Track C if needed.
-

38. Example Settings (Copy/Paste)

- Multi-task run:

```
TrainConfig.use_multi_task_labels = True
TrainConfig.use_ttc_bins = True
TrainConfig.monitor_metric = "mAP"
TrainConfig.eval_every = 1
```

- Eval with priors:

```
EvalConfig.use_hotspot_priors = True
EvalConfig.hotspot_alpha = 0.3
```

```
EvalConfig.use_clip_rerank = True  
EvalConfig.clip_weight = 0.3
```

39. When Things Go Wrong (Debug Steps)

- Set `candidate_limit` low; run tiny batch; print shapes in train loop.
- Run `trackB_train_demo.py` to isolate tokenization vs training issues.
- Inspect `cache/invalid_records.csv` to see why records were skipped.
- Force `val_manifest` to a known file; reduce batch size for eval overlays.
- Disable priors/CLIP to isolate base head performance.

40. Notes on TTC Binning

- Bins: [0,0.5), [0.5,1.0), [1.0,2.0), [2.0, inf).
- `use_ttc_bins=True` switches TTC loss to CE on bins; regression head still present for compatibility.
- Eval `ttc_mode` can be `binned` (use logits) or `reg` (bin the reg prediction).

41. Notes on Positive Definition

- `is_positive` is preferred; falls back to `cls==1`.
- Frame-level metrics assume at most one positive per frame; picks first positive index.
- Ensure manifests mark next-active candidate explicitly for consistent metrics.

42. ROI Pooling Details

- Uses grid-aware mean over token grid inside the bounding box.
- Requires `hw` (grid shape) and `image_size` to map ROI to grid indices.
- Pure Python; no compiled ops; safe on CPU/GPU.

43. Normalization & TTC Stats

- `normalize_ttc=True` z-scores TTC using dataset mean/std.
- Stored in `ds.stats`; used to denorm predictions in eval.
- TTC MAE is computed on denormalized seconds.

44. Candidate Subsampling

- `candidate_limit` in dataset randomly subsamples per sample if too many boxes.
- Track C pruning handles runtime pruning; keep Track B `candidate_limit` to reasonable size for training stability.

45. Stage B Discovery Logic

- `latest_stageB_run` finds newest `trackA_stageB_*` under `runs/Track_A/` containing head manifests.
 - `resolve_stageB_manifest` picks `head_train/val.{jsonl,json}` inside that run.
 - You can override with absolute paths in configs.
-

46. Eval Summary JSON Contents

- `metrics` : full metrics dict.
 - `eval_config` : serialized EvalConfig (paths as strings).
 - Useful for sweeps: compare settings without re-opening code.
-

47. Best Summary JSON Contents (Training)

- `checkpoint` , `monitor_metric` , `monitor_value` , `timestamp` , `metrics` , `train_config` .
 - Filenames match best checkpoint with `_summary` suffix; also mirrored to `trackB_best_summary.json` .
-

48. Plot Summary TSV Columns

- `idx` , `checkpoint` , `timestamp` , then ordered metrics found (accuracy, mAP, TTC, N/Nv/N_delta/All, top-5, priors if present).
 - Easy to diff runs in Excel or `pandas` .
-

49. Comparing Runs Manually

- Use `trackB_metrics_summary.tsv` for quick deltas.
 - For deeper dive, diff `metrics_val_*.json` pairs (baseline vs priors/CLIP).
 - Check predictions JSONL for frames where N/Nv/All differ; open overlays.
-

50. Closing

- Track B is designed to be a small, modular head on top of Track A Stage B outputs.
- This README should let you operate, debug, and extend without re-reading all code.
- If adding features, mirror new toggles and outputs here to keep it the single source of truth.

Track C — RGTP Pruning Evaluation (Detailed Guide)

This README explains the Track C pruning/evaluation pipeline end-to-end. It is deliberately detailed (~200+ lines) so you can run, audit, and extend Track C without digging into the code.

1) Purpose

Track C applies **Run-time Guided Token Pruning (RGTP)** on top of a trained Track B checkpoint. It prunes low-importance candidate tokens before the head, then evaluates accuracy/mAP/TTC and semantic metrics ($N/N+V/N+\delta$), producing both quality metrics and pruning stats (latency/VRAM/FLOPs when enabled).

2) Inputs & Dependencies

- Track B checkpoint: `local_extraction/runs/Track_B/checkpoints/trackB_best.pt` (or any `trackB_final_*.pt` / best variant).
 - Stage B manifests: from Track A Stage B run (`head_val.jsonl/json`) or `local_extraction/v2/manifests/` .
 - Frames: `local_extraction/v2/extracted_frames/<uid>/<frame:07d>.jpg` .
 - Python: torch, tqdm; optional CUDA for latency/VRAM measurements. No network calls.
-

3) Key Files

- `trackC_pruning.py` — main evaluation + pruning harness.
 - `trackC_compare_metrics.py` — compares Track B vs Track C metrics JSON.
 - `trackC_plots.py` — plots Track C metrics over time (includes runtime metrics).
-

4) Runtime Config (edit in `trackC_pruning.py`)

`RuntimeConfig` toggles (no CLI by default):

- Paths: `checkpoint` , `stageB_run` , `val_manifest` (None → auto-discover).
- Pruning: `pruning_enabled` , `rgtp_rate` (fraction to drop), `min_keep` .
- Instrumentation: `measure_latency` , `measure_vram` , `measure_flops` , `bench_warmup` , `bench_iters` , `bench_samples` .

Tips:

- Set `checkpoint` if you want a specific Track B model; otherwise latest/final is auto-picked.
 - Use `pruning_enabled=False` or `rgtp_rate=0` to emulate Track B baseline.
-

5) Evaluation Config (EvalConfig)

- Paths: `frames_root` , `manifests_root` , `trackA_runs_root` , `checkpoint` , `stageB_run` , `val_manifest` .
 - Data: `batch_size` , `candidate_limit` , `normalize_ttc` , `num_workers` (keep 0).
 - Defaults point to `local_extraction/v2` assets and Track B runs/checkpoints.
-

6) RGTP Config (RGTPConfig)

- `enabled` : True/False.
- `rate` : fraction of candidate tokens to prune (clamped [0, 0.95]).
- `min_keep` : minimum candidates to keep per frame.

- `temporal_decay` : blend rollout vs motion energy for importance.

Importance scoring:

- Combines FGTP rollout attention (t-1) with motion energy (t-1 $\rightarrow$ t).
- Each candidate gets a score by ROI-pooling the blended grid.
- Lowest scores are dropped; dropped tokens receive a strong negative logit fill.

7) Instrumentation (InstrumentationConfig)

- `enabled` : set automatically if any of `measure_latency/vram/flops` are True.
- `record_vram` : peak CUDA memory (no-op on CPU).
- `record_flops` : one-off FLOPs profile of head forward pass.
- `use_cuda_events` : latency via CUDA events when available; falls back to wall clock.
- `bench_warmup` , `bench_iters` , `bench_samples` : control micro-benchmark of head forward (`torch.utils.benchmark`).

Runtime outputs (added to metrics JSON):

- Latency stats: mean/median/p90/p95 per sample (ms).
- Throughput: samples/s and candidates/s.
- Head micro-benchmark mean (ms) if enabled.
- Peak VRAM allocated/reserved (bytes) if CUDA.
- Head FLOPs (one-off) if profiling succeeds.

8) Outputs

All under `local_extraction/runs/Track_C/` :

- Metrics: `metrics/trackC_val_rateXX_<timestamp>.json`
 - Accuracy, mAP, TTC MAE, N/N+V/N+ δ /All (top-1), top-5 hit/AP, per-noun/verb stats.
 - Pruning stats: requested rate, mean fraction pruned, counts.
 - Runtime stats (latency/throughput/VRAM/FLOPs) when instrumentation is on.
- Summary: `metrics/trackC_val_rateXX_<timestamp>_summary.json`
 - Contains metrics + serialized eval/pruning/instrumentation configs.
- Plots: from `trackC_plots.py` $\rightarrow$ `plots/trackC_metrics_over_time.png` and TSV summary.

9) How to Run (PowerShell examples)

- Default (pruning on, auto-discover checkpoint/manifest):

```
python local_extraction\trackC\trackC_pruning.py
```

- Disable pruning (baseline timing/metrics):

```
# set RuntimeConfig.pruning_enabled = False or rgtp_rate = 0 in file,
then run
python local_extraction\trackC\trackC_pruning.py
```

- Custom checkpoint/manifest:
 - Set `RuntimeConfig.checkpoint =`
`"local_extraction/runs/Track_B/checkpoints/trackB_best.pt"`
 - Set `RuntimeConfig.val_manifest =`
`"local_extraction/runs/Track_A/<run>/head_val.jsonl"`

10) Internal Flow (trackC_pruning.py)

1. Resolve Stage B run/val manifest (latest Track A Stage B if not provided).
 2. Resolve Track B checkpoint (explicit or latest `trackB_final_*.pt`).
 3. Load projector/fusion/head; infer class vocab and optional heads from checkpoint weights.
 4. Build `TrackBDataset` (val) with tokenizer; DataLoader (num_workers=0).
 5. For each sample:
 - Project img/vid tokens → fused tokens (via Track B fusion).
 - ROI-pool per candidate.
 - Compute RGTP scores (rollout + motion), build keep mask; drop low-score tokens.
 - Run head on kept tokens; fill dropped positions with negative logits/zeros (and multi-task logits if present).
 - Collect logits, TTC (denorm), labels, semantics; update frame-level N/N+V/N+ δ metrics if heads present.
 - Record latency/bench/FLOPs/VRAM if enabled.
 6. Aggregate metrics; compute prune rate; attach runtime stats; include checkpoint/manifest paths.
 7. Save metrics JSON and summary JSON; print key metrics.
-

11) Metrics Tracked

- Candidate-level: `accuracy` , `mAP` , `ttc_mae_seconds` , `num_candidates` .
 - Pruning: `rgtp_enabled` , `rgtp_rate_request` , `rgtp_mean_fraction_pruned` , `samples_total` , `samples_without_candidates` .
 - Frame-level (top-1): `N_mAP` , `Nv_mAP` , `N_delta_mAP` , `All_mAP` (when noun/verb/bin available).
 - Frame-level (top-5 hit): `N_top5_acc` , `Nv_top5_acc` , `N_delta_top5_acc` , `All_top5_acc` .
 - Frame-level (top-5 AP): `N_top5_mAP` , `Nv_top5_mAP` , `N_delta_top5_mAP` , `All_top5_mAP` .
 - Per-class breakdowns: `per_noun_stats` , `per_verb_stats` .
 - Runtime: `latency_ms_mean` , `latency_ms_median` , `latency_ms_p90` , `latency_ms_p95` , `throughput_samples_per_s` , `throughput_candidates_per_s` , `head_benchmark_ms_mean` , `peak_vram_bytes` , `peak_vram_reserved_bytes` , `head_flops` (when available).
-

12) Plotting (trackC_plots.py)

- Scans `metrics/trackC_val_rate*.json` .
- Plots all important scalar metrics including runtime fields (latency/throughput/VRAM/FLOPs) when present.
- Writes:
 - `plots/trackC_metrics_over_time.png`
 - `plots/trackC_metrics_summary.tsv` (tabular snapshot with metric columns found).
- Usage:

```
python local_extraction\trackC\trackC_plots.py --runs_dir
local_extraction/runs/Track_C --out_dir
local_extraction/runs/Track_C/plots
```

13) Comparing Track B vs Track C

- Script: `trackC_compare_metrics.py`
 - Defaults: picks latest `metrics_val_*.json` (Track B) and `trackC_val_rate*.json` (Track C).
 - Prints side-by-side table for key metrics (accuracy, mAP, TTC, N/Nv/Nδ/All, top-5).
 - Override with `--b_metrics` / `--c_metrics` to compare specific runs.
-

14) Common Workflows

- **Baseline vs Pruned:** run once with pruning off (`pruning_enabled=False`), once with pruning on (e.g., `rgtp_rate=0.5`), compare `N_mAP` , `mAP` , and latency stats.
 - **Rate Sweep:** edit `RuntimeConfig.rgtp_rate` across runs (e.g., 0.25/0.5/0.75), then use plots/compare scripts to build mAP-latency curves.
 - **Multi-task Semantics:** if Track B checkpoint has noun/verb/bin heads, check `Nv_mAP` , `All_mAP` to ensure pruning preserves semantics.
 - **Latency Focus:** enable instrumentation; watch `latency_ms_*` and throughput vs `rgtp_mean_fraction_pruned` .
-

15) Troubleshooting

- **No val manifest found:** set `RuntimeConfig.val_manifest` explicitly; ensure Stage B run exists or manifests root has `head_val*.json[1]` .
- **Checkpoint missing:** set `RuntimeConfig.checkpoint` to an existing Track B checkpoint; otherwise train Track B first.
- **CUDA OOM:** reduce `candidate_limit` , disable pruning (to debug), or run on CPU; pruning itself should reduce head load when rate > 0.
- **Latency not recorded:** ensure `measure_latency=True` ; CUDA events require CUDA; CPU falls back to wall clock.
- **VRAM fields zero:** likely CPU run; set `measure_vram=False` if CPU-only to avoid confusion.
- **FLOPs missing:** set `measure_flops=True` and ensure a CUDA-capable build; profiling may be expensive.

16) RGTP Scoring Details

- Rollout: uses fusion FGTP attention to score spatial tokens at t-1.
 - Motion: squared difference between last two temporal slices.
 - Blend: `temporal_decay * rollout + (1 - temporal_decay) * motion`.
 - ROI pooling: averages blended grid over each candidate box → score.
 - Pruning: keep top-K by score, where `K = max(min_keep, round(N * (1 - rate)))`.
 - Dropped candidates: assigned `LOGIT_FILL` (very negative) so their next-active prob is near zero; TTC set to zero; multi-task logits filled likewise.
-

17) Frame-Level Semantic Metrics in Track C

- Requires Track B checkpoint with noun/verb heads (and TTC bin head if `ttc_mode='binned'`):
 - `N_mAP` : noun correct + IoU >= 0.5 on top candidate.
 - `Nv_mAP` : noun+verb correct + IoU >= 0.5.
 - `N_delta_mAP` : noun + TTC bin correct + IoU >= 0.5 (bin from logits or reg → bin).
 - `All_mAP` : noun+verb+TTC bin correct + IoU >= 0.5.
 - Top-5 versions treat top-5 candidates per frame as predictions for AP/hit metrics.
-

18) File Map

- `trackC_pruning.py` : main eval + pruning + instrumentation + metrics writing.
 - `trackC_compare_metrics.py` : diff Track B vs Track C metrics.
 - `trackC_plots.py` : plot Track C metrics including runtime fields.
 - Metrics dir: `runs/Track_C/metrics/`.
 - Plots dir: `runs/Track_C/plots/` (created by `trackC_plots.py`).
-

19) Sample Metrics JSON Excerpt

```
{
  "accuracy": 0.59,
  "mAP": 0.33,
  "ttc_mae_seconds": 0.19,
  "rgtp_mean_fraction_pruned": 0.50,
  "N_mAP": 0.25,
  "Nv_mAP": 0.05,
  "N_delta_mAP": 0.22,
  "latency_ms_mean": 12.5,
  "throughput_samples_per_s": 80.0
}
```

20) Tips for Reproducibility

- Use descriptive run names for Track B checkpoints and Track A Stage B runs.
 - Keep pruning rates and instrumentation toggles in the summary JSON to trace exact settings.
 - For sweeps, script multiple runs changing only `rgtp_rate` ; then plot with `trackC_plots.py` .
-

21) Notes on Candidates & Pruning Interaction

- `candidate_limit` (EvalConfig/Dataset) caps candidates before pruning; RGTP prunes after scoring.
 - If rate is high and `min_keep` low, many candidates get `LOGIT_FILL` ; metrics still count them but with near-zero prob.
 - If a sample has 0 candidates, it is skipped and counted in `samples_without_candidates` .
-

22) Hand-off from Track B

- Track C loads Track B checkpoint and infers head shapes (noun/verb/bin) from weight shapes.
 - Uses Track B tokenizer/fusion/head directly; no retraining.
 - `ttc_mode` defaults to `bin` if bin head present, else `reg` .
-

23) Overlay Generation (Why Track C doesn't do it)

- Track C focuses on pruning/time vs quality; overlays are already produced in Track B eval.
 - If you need overlays under pruning, you can adapt Track B eval to apply RGTP scoring before head forward; Track C currently omits overlays to keep runtime slim.
-

24) Extending Track C

- Different pruning strategies: swap scoring function (`_candidate_scores`).
 - Structured pruning: keep top-K per grid region; requires changing ROI pooling strategy.
 - Multi-head latency analysis: add more granular timers around tokenizer/fusion/head.
 - Batch-mode pruning: vectorize pruning across batch for speed (currently per-sample).
-

25) Cleanup & Storage

- Metrics/plots accumulate under `runs/Track_C/` ; delete old runs to save disk.
 - Runtime summaries are duplicated in summary JSON; plots pick up new fields automatically.
-

26) Glossary

- RGTP: Run-time Guided Token Pruning.
 - LOGIT_FILL: very negative logit assigned to pruned candidates.
 - N/Nv/Nδ/All: noun; noun+verb; noun+TTC-bin; noun+verb+TTC-bin metrics at frame level.
 - TTC: Time-to-contact (seconds).
 - FGTP: Frame-Guided Temporal Pooling (from Track B fusion).
-

27) Minimal Checklist

- ☐ Track B checkpoint available.
- ☐ Val manifest resolvable (Stage B run or explicit path).
- ☐ Pruning toggles set (`pruning_enabled` , `rgtp_rate` , `min_keep`).
- ☐ Instrumentation toggles set if latency/VRAM/FLOPs needed.
- ☐ Run script; confirm metrics + summary JSON.
- ☐ Plot metrics if comparing multiple runs.

This README should give you everything needed to operate and extend Track C locally. Trim sections if you want a shorter version for your own notes.